

PyTrackingAnalysis — User Guide

Table of contents

- 1. Overview
- 2. Environment setup with uv
 - Windows
 - macOS
 - Linux
- 3. Project directory structure
- 4. The tracking_config.yaml reference
- 5. The desktop UI
 - Launching the UI
 - Config tab
 - Run tab
 - Outputs tab
- 6. Running the pipeline from a notebook or script
- 7. Understanding the outputs
- 8. Batch analysis across multiple experiments
- 9. Quick reference

1. Overview

PyTrackingAnalysis is a Python pipeline for analysing insect-tracking data exported from DTrack. A single configuration file (`tracking_config.yaml`) describes the experiment — the tracking hardware, the experimental design, and how each physical tracking region maps to a treatment group. From that one file the pipeline can produce summary CSVs, statistical comparisons, publication-quality plots, and a multi-page PDF report.

There are three ways to drive the pipeline:

Interface	Best for
Desktop UI (<code>analysis_ui.py</code>)	Day-to-day use; no Python knowledge needed
Jupyter notebook (<code>Notebooks/SimpleTracker.ipynb</code>)	Exploratory work; custom plots
Python script / REPL	Automation; integration with other tools

2. Environment setup with uv

`uv` is a fast Python package manager that reads `pyproject.toml` and locks exact dependency versions. The project requires **Python 3.13 or later**.

2.1 Windows

Install uv

Open PowerShell and run:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1  
| iex"
```

Restart the terminal so `uv` is on `PATH`.

Clone the repository and create the environment

```
git clone https://github.com/your-org/PyTrackingAnalysis.git  
cd PyTrackingAnalysis  
  
# Create a virtual environment using the Python version in .python-version  
(3.13)  
uv sync
```

`uv sync` reads `pyproject.toml`, downloads and installs all dependencies into `.venv\`, and is idempotent — safe to run again after pulling updates.

Activate the environment (optional — `uv run` works without it)

```
.venv\Scripts\Activate.ps1
```

Run the UI

```
# With the environment activated:  
python analysis_ui.py  
  
# Or without activating, using uv run:  
uv run python analysis_ui.py  
  
# With an explicit project directory:  
uv run python analysis_ui.py "C:\Users\you\Experiments\Trial1"
```

PyQt6 note on Windows: If a "platform plugin not found" error appears, install the Visual C++ Redistributable from Microsoft's website, then retry.

2.2 macOS

Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Restart the terminal (or run `source ~/.zshrc` / `source ~/.bash_profile`).

Clone the repository and create the environment

```
git clone https://github.com/your-org/PyTrackingAnalysis.git
cd PyTrackingAnalysis
uv sync
```

Activate the environment (optional)

```
source .venv/bin/activate
```

Run the UI

```
# With environment activated:
python analysis_ui.py

# Without activating:
uv run python analysis_ui.py

# With a project directory:
uv run python analysis_ui.py /path/to/Trial1
```

macOS display server note: PyQt6 requires a display. Confirm XQuartz is not needed (native macOS Cocoa backend is used automatically).

2.3 Linux

Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
# Or via pip if curl is unavailable:
pip install uv
```

Restart the terminal or run `source ~/.bashrc`.

Install system Qt dependencies (if missing)

On Debian/Ubuntu:

```
sudo apt install libxcb-cursor0 libxcb-xinerama0 libxkbcommon-x11-0
```

On Fedora/RHEL:

```
sudo dnf install xcb-util-cursor libxkbcommon-x11
```

Clone the repository and create the environment

```
git clone https://github.com/your-org/PyTrackingAnalysis.git
cd PyTrackingAnalysis
uv sync
```

Activate the environment (optional)

```
source .venv/bin/activate
```

Run the UI

```
python analysis_ui.py           # cwd must be the repo root
uv run python analysis_ui.py     # works from any directory
uv run python analysis_ui.py /path/to/Trial1 # open a specific project
```

Headless / SSH note: PyQt6 requires an X11 or Wayland display. If running over SSH, forward the display with `ssh -X` or use a VNC/RDP session. For fully headless batch processing, use the Python API (§6) instead.

Updating the environment

After pulling new commits:

```
git pull
uv sync           # installs / removes packages to match the updated lock
file
```

3. Project directory structure

A **project directory** is the root folder for a single experiment. It must contain `tracking_config.yaml` and a `data/` sub-folder with the DTrack export files. The pipeline creates `analysis/` and `qc/`

automatically on first run.

```

MyExperiment/                                ← project directory (pass this to the
UI)
├── tracking_config.yaml                     ← experiment configuration (required)
├── data/                                   ← DTrack export files (required)
│   ├── ExperimentName.xlsx                 ← main DTrack workbook
│   ├── ExperimentName_Data_1.csv           ← per-tracker CSV, one per tracking
region
│   ├── ExperimentName_Data_2.csv
│   ├── ...
│   ├── ExperimentName_BG.jpg              ← background image (optional)
│   ├── LeftProgram.txt                    ← stimulus program files (optional)
│   └── RightProgram.txt
├── analysis/                              ← created by the pipeline
│   ├── ExperimentName_experiment_summary.txt
│   ├── ExperimentName_Summary.csv
│   ├── ExperimentName_Summary_Facet.csv
│   ├── ExperimentName_Stats.txt
│   ├── ExperimentName_report.pdf
│   ├── ExperimentName_plot_pi_facet.png
│   ├── ExperimentName_plot_percentage_facet.png
│   ├── ExperimentName_plot_transitions_facet.png
│   └── ExperimentName_plot_totaldistance_facet.png
└── qc/                                    ← created by the pipeline
    └── ExperimentName_data_quality.csv

```

Naming rules:

- The `.xlsx` file determines the experiment name. Every output file is prefixed with that name.
- The per-tracker CSVs must follow the pattern `<name>_Data_<N>.csv`. `N` is matched to tracking region `T_N-1` in the YAML (i.e. `_Data_1.csv` → region `T_0`).
- `tracking_config.yaml` must be at the **top level** of the project directory — not inside `data/`.

Batch layout (multiple experiments)

For batch analysis, place multiple project directories under a common parent:

```

AllExperiments/                             ← pass this as the "Batch parent" in
the UI
├── Trial1/
│   ├── tracking_config.yaml
│   └── data/ ...
├── Trial2/
│   ├── tracking_config.yaml
│   └── data/ ...
└── Trial3/
    ├── tracking_config.yaml
    └── data/ ...

```

Each sub-directory is processed independently. Directories that lack either `tracking_config.yaml` or a `.xlsx` file inside `data/` are silently skipped.

4. The tracking_config.yaml reference

`tracking_config.yaml` has three top-level sections: `global`, `tracking_regions`, and `counting_regions`. Only `global` and `tracking_regions` are required.

4.1 global — required fields

```
global:
  tracking_type: TWOCHOICETRACKER    # see table below
  tracking_rig:  colosseum           # see table below
```

tracking_type

Selects the analysis mode. Choose the one that matches how DTrack recorded your data.

Value	Description
TRACKER	Position tracking only — computes distance and speed
TWOCHOICETRACKER	Two-region choice assay — computes PI, percentage time, transitions
XCHOICETRACKER	Multi-region assay along a linear axis — computes adjusted X position
PAIRWISEINTERACTIONTRACKER	Proximity-based interaction scoring between pairs of tracked animals
COUNTER	Frame-count based occupancy (no continuous position)
TWOCHOICECOUNTER	Counter-based two-region choice — PI and percentage
PAIRWISEINTERACTIONCOUNTER	Counter-based pairwise interaction scoring

tracking_rig

Selects the hardware calibration preset. `fps` and `mm_per_pixel` are set automatically from the preset; all other parameters use the defaults shown below.

Value	mm per pixel	Notes
small_arena	0.056	
arena_max	0.145	
colosseum	0.108	
obscura	0.131	

Value	mm per pixel	Notes
movie	—	You must supply <code>fps</code> and <code>mm_per_pixel</code> manually

`fps` for all hardware rigs is read from the timestamps in the DTrack export rather than set to a fixed value. Only the `movie` rig requires an explicit `fps`.

4.2 global — experimental design

```
global:
  experimental_design_factors:
    feeding: [Starved, Control]
    sex:      [Male, Female]
```

List every factor and its levels. These are used for axis labels and plot titles. The actual assignment of factors to each physical tracking region is made in `tracking_regions` (see §4.3).

4.3 global — optional fields

```
global:
  # Split the recording into time phases for faceted plots and statistics.
  # [10, 70] creates three phases: 0-10 min, 10-70 min, 70+ min.
  # Remove this key entirely to disable faceted analysis.
  facet_cutoffs: [10, 70]

  # Only needed for the 'movie' rig, or to override a hardware preset.
  fps: 30
  mm_per_pixel: 0.108

  # Smoothing window for speed calculation (seconds). Default: 1
  speed_window_seconds: 1

  # Speed range [min, max] mm/s that defines micro-movement. Default: [0.2, 2]
  micromove_speed_mm_sec: [0.2, 2]

  # Minimum speed (mm/s) to count as walking. Default: 2
  walking_speed_mm_sec: 2

  # Minimum continuous resting duration (minutes) to count as sleep.
  # Default: 5
  sleep_threshold_min: 5

  # Distance thresholds (mm) for interaction detection.
  # Only used with PAIRWISEINTERACTIONTRACKER / PAIRWISEINTERACTIONCOUNTER.
  interaction_distances: [8]
```

4.4 tracking_regions

One entry per DTrack tracking region. Region names must match the naming scheme used in the CSV files: region `T_0` corresponds to `_Data_1.csv`, `T_1` to `_Data_2.csv`, and so on.

```
tracking_regions:
  T_0:
    experimental_factors: Starved, Female # comma-separated, order must
    match                                     # experimental_design_factors
    order
    x_location_multiplier: 1                # set to -1 to flip the X axis
    y_location_multiplier: 1                # set to -1 to flip the Y axis
  T_1:
    experimental_factors: Control, Male
    x_location_multiplier: 1
    y_location_multiplier: 1
  # ... one entry per region
```

`x_location_multiplier` and `y_location_multiplier` correct for physical differences in camera orientation between regions. Use `1` for no correction and `-1` to mirror an axis.

4.5 counting_regions (counter types only)

Required when `tracking_type` is `COUNTER`, `TWOCHOICECOUNTER`, or `PAIRWISEINTERACTIONCOUNTER`. Maps the region labels used inside the DTrack `.xlsx` workbook to canonical treatment names.

```
counting_regions:
  Light:
    alias: Light, LL, L # any of these strings in the workbook =
    "Light"
  NoLight:
    alias: NoLight, NL, N
```

`TWOCHOICECOUNTER` must have **exactly two** counting regions.

4.6 Complete minimal example

```
global:
  tracking_type: TWOCHOICETRACKER
  tracking_rig: colosseum
  experimental_design_factors:
    genotype: [WT, Mutant]
  facet_cutoffs: [10, 60]
```



```
tracking_regions:
  T_0:
    experimental_factors: WT
    x_location_multiplier: 1
    y_location_multiplier: 1
  T_1:
    experimental_factors: Mutant
    x_location_multiplier: 1
    y_location_multiplier: 1
```

5. The desktop UI

The desktop UI ([analysis_ui.py](#)) provides three tabs — **Config**, **Run**, and **Outputs** — all anchored to a single project directory chosen at the top.

5.1 Launching the UI

```
# From the repository root with the uv environment active:
python analysis_ui.py

# Open directly to a specific project directory:
python analysis_ui.py /path/to/MyExperiment

# Without activating the environment:
uv run python analysis_ui.py /path/to/MyExperiment
```

The **Project directory** bar at the top of the window determines where the UI looks for [tracking_config.yaml](#) and where it seeds the run-path fields. Use **Choose...** to browse, or type a path directly and press Enter. **Reload** re-reads the config file and refreshes the output file list.

5.2 Config tab

The Config tab provides two ways to edit [tracking_config.yaml](#):

Form sub-tab

A structured, point-and-click interface organised into three inner tabs:

- **Global** — drop-downs for [tracking_type](#) and [tracking_rig](#); an editable table for experimental design factors; optional facet cutoffs; and text fields for all parameter overrides.
- **Tracking regions** — a table with one row per region. The **Generate N regions** button creates [T_0](#) through [T_\(N-1\)](#) in one click, ready to fill in. X/Y multipliers are drop-downs restricted to [1](#) and [-1](#).
- **Counting regions** — a table mapping canonical treatment names to their DTrack aliases.

Use **← Load values from YAML into Form** to populate the form from whatever is currently in the YAML editor (useful after manually editing the YAML).

YAML sub-tab

A plain-text editor showing the raw YAML. Useful for:

- Copying/pasting a config from another source
- Making quick edits while preserving comments
- Checking exactly what will be written to disk before saving

Use **← Load values from Form into YAML** to regenerate the YAML from the current form state. Note that this removes any hand-written YAML comments.

Shared buttons

Button	Action
Save config	Writes to <code>tracking_config.yaml</code> inside the project directory. When the Form sub-tab is active, the form is first dumped to YAML, then saved (comments are not preserved). When the YAML sub-tab is active, the raw text is saved as-is, preserving comments.
Validate YAML	Checks that the current text or form output is syntactically valid YAML. Does not check semantic correctness.
Reload from file	Discards any unsaved edits and re-reads from disk.

If you click **Run Single Analysis** or **Run Batch Analysis** with unsaved edits, the UI will prompt you to Save, Discard, or Cancel before proceeding.

5.3 Run tab

Analysis paths

Field	What it means
Single project	A project directory containing <code>tracking_config.yaml</code> and <code>data/</code> . Seeded from the global project directory; change it independently with Browse... to run a different experiment without altering the global path.
Batch parent	The parent directory that contains multiple project directories as sub-folders. Each sub-folder must have its own <code>tracking_config.yaml</code> and <code>data/</code> folder.

Both fields are independently editable. The global project directory at the top only seeds them on first load; changes to the run-tab fields do not update the global directory.

Running an analysis

Click **Run Single Analysis** or **Run Batch Analysis**. While the analysis runs:

- Both buttons are disabled.

- A progress bar is displayed below the buttons.
- The **Execution log** streams output in real time, line by line.

The analysis runs in a background thread so the UI remains responsive. When complete, the Outputs tab is automatically refreshed.

A failed run prints the full Python traceback to the log and shows a brief error dialog.

5.4 Outputs tab

After an analysis completes, the Outputs tab lists every file produced under the project directory (and any batch sub-directories) in `analysis/` and `qc/`.

File list

Files are shown as `subfolder/filename` for files at the top level, and as `TrialN/subfolder/filename` for files in batch sub-directories. Click any file to preview it.

Preview pane

File type	Preview
<code>.txt</code> , <code>.csv</code> , <code>.yaml</code> , <code>.log</code>	Plain-text view
<code>.png</code> , <code>.jpg</code> , <code>.jpeg</code> , <code>.bmp</code>	Inline image (scrollable)
<code>.pdf</code> , others	"Open externally" — use the button below

Buttons

- **Refresh** — re-scans the output directories without re-running the analysis.
- **Open externally** — opens the selected file in the default system application (PDF viewer, image viewer, spreadsheet application, etc.).

6. Running the pipeline from a notebook or script

The analysis pipeline is also available as a Python API. This is the approach used in `Notebooks/SimpleTracker.ipynb`.

Setup

```
import os
# If running from the Notebooks/ sub-directory, step up to the repo root
first:
os.chdir("../")

import warnings
warnings.filterwarnings("ignore")
```

```
from Experiment import Experiment, batch_analyze
```

Single experiment

```
# Pass the project directory – the one containing tracking_config.yaml and
data/
exp = Experiment("./Data/Trial1/")

# Human-readable summary of the loaded experiment
print(exp)

# Detailed per-tracker overview (also saved to analysis/)
exp.experiment_summary()

# Data quality report (also saved to qc/)
exp.qc(cutoff=0.9)      # cutoff = minimum fraction of valid frames per
tracker

# Save summary CSVs (flat + faceted) to analysis/
exp.save_summary()

# Run pairwise statistical comparisons and save results to analysis/
exp.stats()

# Save plots to analysis/ as PNG files
exp.save_plots()

# Build a multi-page PDF report in analysis/
exp.create_report()

# — OR run the complete pipeline in one call:
_____
exp.run_analysis()      # summary → qc → save_summary → save_plots → stats
exp.create_report()    # PDF report (separate call so you can skip it)
```

Accessing individual plots interactively

Inside a Jupyter notebook with `%matplotlib inline`:

```
# These display the plot inline in the notebook (not saved to disk).
exp.arena.plot_pi()
exp.arena.plot_pi_facet(cutoffs=[10, 70])
exp.arena.plot_percentage_facet(cutoffs=[10, 70])
exp.arena.plot_transitions_facet(cutoffs=[10, 70])
exp.arena.plot_totaldistance_facet(cutoffs=[10, 70])
```

The available plot methods depend on `tracking_type`:

tracking_type	Plot methods
TRACKER	plot_totaldistance_facet
TWOCHOICETRACKER	plot_pi_facet, plot_percentage_facet, plot_transitions_facet, plot_totaldistance_facet
TWOCHOICECOUNTER	plot_pi_facet, plot_percentage_facet
XCHOICETRACKER	plot_adjusted_x_position_facet, plot_totaldistance_facet
PAIRWISEINTERACTIONTRACKER	plot_interactions_facet, plot_totaldistance_facet
PAIRWISEINTERACTIONCOUNTER	plot_interactions_facet

Batch processing from a script

```
results = batch_analyze("./Data/")

for path, status in results.items():
    tag = "OK " if status == "ok" else "FAIL"
    print(f" {tag} {path}")
    if status != "ok":
        print(f"      {status}")
```

`batch_analyze` scans every immediate sub-directory of the supplied path, identifies valid experiment directories, runs `exp.run_analysis()` and `exp.create_report()` on each, and returns a `{path: "ok" | error_message}` dictionary.

7. Understanding the outputs

All outputs are written relative to the project directory.

`analysis/` — main results

File	Contents
*_experiment_summary.txt	Rig settings, tracker count, data quality overview, time range
*_Summary.csv	Per-tracker summary statistics (one row per tracker)
*_Summary_Facet.csv	Same, split into the time phases defined by <code>facet_cutoffs</code>
*_Stats.txt	Pairwise statistical comparisons (Mann-Whitney U) across treatment groups
plot.png	One PNG per plot type, named after the plot method
*_report.pdf	Multi-page PDF: experiment summary → QC table → tracker grid plots → all plots

qc/ — data quality

File	Contents
*_data_quality.csv	Per-tracker fraction of valid (non-missing) frames; trackers below cutoff are flagged

8. Batch analysis across multiple experiments

Requirement

Each experiment sub-directory must be a self-contained project directory:

```

ParentFolder/
├── Experiment_A/
│   ├── tracking_config.yaml    ← required
│   └── data/
│       ├── Experiment_A.xlsx  ← required
│       └── Experiment_A_Data_*.csv
└── Experiment_B/
    ├── tracking_config.yaml
    └── data/
        ├── Experiment_B.xlsx
        └── Experiment_B_Data_*.csv

```

Running in the UI

1. Set the **Batch parent** field in the Run tab to **ParentFolder/**.
2. Click **Run Batch Analysis**.
3. Results appear in each experiment's own **analysis/** and **qc/** folders.
4. The Outputs tab shows files from all experiments, prefixed with the sub-directory name (e.g. **Experiment_A/analysis/Experiment_A_report.pdf**).

Running from Python

```

from Experiment import batch_analyze

results = batch_analyze("./ParentFolder/")

```

Notes

- Each sub-directory uses its **own** **tracking_config.yaml**, so different experiments can have different tracking types, rigs, and experimental designs.
- Sub-directories that are missing **tracking_config.yaml** or a **.xlsx** file inside **data/** are silently skipped.
- A failure in one experiment does not stop the others.

9. Quick reference

Starting the UI

```
# From repo root:
python analysis_ui.py
python analysis_ui.py /absolute/path/to/project

# Via uv (works from any directory):
uv run python analysis_ui.py /absolute/path/to/project

# Structured config editor (standalone):
python config_ui.py
python config_ui.py /path/to/project
```

Minimal tracking_config.yaml

```
global:
  tracking_type: TWOCHOICETRACKER
  tracking_rig: colosseum
  experimental_design_factors:
    treatment: [A, B]

tracking_regions:
  T_0:
    experimental_factors: A
    x_location_multiplier: 1
    y_location_multiplier: 1
  T_1:
    experimental_factors: B
    x_location_multiplier: 1
    y_location_multiplier: 1
```

One-line pipeline (Python)

```
from Experiment import Experiment
exp = Experiment("/path/to/project/")
exp.run_analysis()
exp.create_report()
```

Rig calibration values

tracking_rig	mm / pixel
small_arena	0.056

tracking_rig	mm / pixel
arena_max	0.145
colosseum	0.108
obscura	0.131
movie	user-supplied

Default analysis parameters

Parameter	Default	YAML key
Speed smoothing window	1 second	speed_window_seconds
Micro-movement range	0.2 – 2 mm/s	micromove_speed_mm_sec
Walking threshold	2 mm/s	walking_speed_mm_sec
Sleep threshold	5 min continuous rest	sleep_threshold_min
Interaction distance	8 mm	interaction_distances

Environment commands

```
uv sync                # install / update all dependencies
uv add <package>        # add a new dependency
uv run python <script>   # run without activating the environment
source .venv/bin/activate # activate on macOS / Linux
.venv\Scripts\Activate.ps1 # activate on Windows PowerShell
```